

J-ART: A Japanese Adversarial Red-Team Framework for Application-Layer LLM Security and Cost-Efficiency

Anonymous authors

Paper under double-blind review

Abstract

Most open red-teaming and jailbreak benchmarks for large language models (LLMs) are English-centric and evaluate *bare models*. We present **J-ART** (Japanese Adversarial Red-Team framework), an open, reproducible harness that (i) evaluates the **application-structure layer** — model $\times$ system-prompt strength $\times$ guardrail $\times$ retrieval (RAG) — rather than bare models; (ii) applies **Japanese-specific obfuscation transforms** (gyaru-moji character substitution, vertical/newline splitting, keigo “polite-coercion” framing, double-talk false premises, Base64, and leet/zero-width smuggling) that defeat naïve keyword filters; (iii) grounds attacks in **MITRE ATLAS** (9 techniques, all at inference time); and (iv) reports a **cost-efficiency** metric alongside a defense rate with **Wilson 95% confidence intervals**. Attacks use *harmless proxy markers* (a fictitious canary and benign output markers); the framework contains **no weaponizable payloads**, and the malicious “core” of each attack is masked in all published artifacts. Across 21 application configurations and 11 attacks $\times$ 7 transforms — a live $K = 5$ campaign for the rates and a balanced $K = 3$ ablation on **three** base models for the difference claims — we find that **application-layer configuration collapses the large naked-model spread**: naked models range 29.6–86.5%, yet every hardened or guardrailed configuration on all three ablated models lands in the **91–100%** band, so model choice matters far less once the application is configured. With **cluster-robust** intervals, a **hardened prompt and model-based guardrails significantly raise defense on each ablated model**, while lightweight keyword/regex filters help most where the base model is weak (and are not individually significant near the ceiling, where the hardened arm saturates). Defense rates are **lexical-leak upper bounds** that cannot fully separate genuine refusal from marker-suppression (§7). A simple **normalizing filter** defeats the obfuscation transforms a naïve keyword filter misses. We also document **large within-campaign variance**: in a controlled $K = 5$ repeat of a frozen configuration set, **9.6% of cells changed outcome across identical trials**, which makes single-sample leaderboards unreliable and motivates our statistical treatment.

Note. This is *not* an official safety evaluation of any vendor’s model; reported numbers are run-time snapshots subject to Section 7, and all defense rates are *lexical-leak* upper bounds (Section 3). The Section 4 numbers reproduce from committed result artifacts; for double-blind review, the code and artifacts are provided through an anonymized repository (Section 9) and will be released under open licenses upon acceptance.

1 Introduction

Public LLM red-teaming tooling (e.g., garak [2]) and jailbreak benchmarks (e.g., HarmBench [3], Jailbreak-Bench [4]) are predominantly English. Multilingual safety work has shown that *translating* prompts into low-resource languages can bypass safety training, but **language-specific obfuscation** — exploiting a language’s own scripts, typography, and register — remains comparatively underexplored. Japanese in particular offers a rich obfuscation surface (multiple scripts, full-width/zero-width characters, vertical writing, and conventionalized polite registers).

Two further gaps motivate J-ART. First, **bare-model versus deployed-system evaluation**: practitioners deploy a model *plus* a system prompt, *plus* guardrails, often *plus* retrieval; the security-relevant unit is the **application configuration**, not the model. Second, **security and cost jointly**: defenses cost tokens or latency, so we report a cost-efficiency score so that “cheapest sufficiently-safe configuration” is directly readable.

Our contributions are: (a) a Japanese obfuscation transform suite; (b) an application-layer, MITRE-ATLAS-grounded harness with a safe proxy-marker methodology; (c) a guardrail comparison including a normalizing filter and a model-based classifier; (d) a statistically-reported leaderboard (Wilson CIs, optional repeated trials) with a cost-efficiency metric; and (e) an empirical observation of large run-to-run variance induced by model routers (which dispatch each request to an actual serving backend).

2 Related Work

J-ART sits at the intersection of six lines of work, summarized below and positioned in Table 1.

Adversarial red-teaming tooling. garak [2] scans a *model* with a library of vulnerability probes. J-ART instead treats the deployed *application configuration* (model $\times$ system prompt $\times$ guardrail $\times$ RAG) as the unit under test.

Jailbreak and harmful-behavior benchmarks. AdvBench/GCG [11], Jailbroken [12], HarmBench [3], JailbreakBench [4], and in-the-wild prompts [13] standardize *which* harmful behaviors to elicit and *how* to score them, predominantly in English at the model layer. J-ART reuses the harness idea but replaces harmful elicitation with a safe proxy-marker objective (§3) so the suite can be released publicly.

Automated attack generation. PAIR [6] and TAP [7] optimize jailbreak prompts against a target. J-ART uses a *fixed, auditable* transform suite, trading attack strength for reproducibility and safe disclosure.

Multilingual and low-resource safety. Deng et al. [14] and Yong et al. [5] show alignment degrades under *translation* into low-resource languages, and Wang et al. [15] study cross-lingual safety generalization. These study *translation across* languages; J-ART studies *intra-language orthographic obfuscation* specific to Japanese (gyaru-moji, vertical writing, keigo framing, kana/kanji encoding), which persists even though Japanese is high-resource.

Encoding/typographic obfuscation. Encoding-based jailbreaks are established prior art: Cipher-Chat [28] (ciphered conversation) and ArtPrompt [29] (ASCII-art typography), and Boucher et al. [34] (“Bad Characters”) is the canonical zero-width/homoglyph perturbation attack — the nearest prior art to our zero-width smuggling in `gyaru/leet_smuggle`. Their perturbations target classifier/MT models *im-perceptibly*, whereas gyaru-moji is a *human-legible*, culturally-conventionalized Japanese script substitution. Our `base64_wrap` and `leet_smuggle` transforms are **language-agnostic replications** of this line, used only as a baseline; the novel surface is the Japanese-script transforms (§3).

Prompt injection and application-layer evaluation. Perez & Ribeiro [16] introduced goal-hijacking; Greshake et al. [8] formalized *indirect* injection; Liu et al. [17] benchmark LLM-integrated apps; Agent-Dojo [30] is the closest application-layer evaluation (tool-using agents). J-ART instead holds the *application configuration* (prompt $\times$ guardrail $\times$ RAG) as the explicit unit of evaluation against a Japanese-obfuscation surface.

Guardrails and input classifiers. Llama Guard [9], NeMo Guardrails [18], and the OpenAI Moderation API [19] provide model- or rule-based filtering. J-ART treats the guardrail as a *swappable configuration variable* and measures the marginal defense each adds, including a deterministic normalizing filter that defeats obfuscations a naive keyword filter misses (§4).

Table 1: Positioning of J-ART relative to representative prior work.

Work	Language	Unit of eval.	Cost axis	Guard/RAG	Public
garak [2]	English	Model	No	No	Yes
HarmBench	English	Model	No	No	Yes
JBB [3; 4]					
PAIR / TAP [6; 7]	English	Model	No	No	Partial
Multiling. [5; 14; 15]	Low-resource	Model	No	No	Yes
Llama Guard / NeMo [9; 18]	Eng.-centric	Classifier	No	n/a	Yes
J-ART	Japanese	App config	Yes	Yes	On accept.

LLM-as-judge validation. Zheng et al. [20] and the survey of Gu et al. [21] document the biases of LLM judges. J-ART avoids an LLM judge for its *primary* outcome, using a deterministic canary/marker check (§3) so judge variance does not enter the headline metric.

Japanese NLP resources and taxonomies. General Japanese benchmarks such as JGLUE [22] cover NLU, but to our knowledge no open, reproducible *adversarial-safety* benchmark targets Japanese-specific obfuscation — the gap J-ART addresses. We map attacks to MITRE ATLAS [1] and cross-reference the OWASP Top 10 for LLM Applications [10].

J-ART is thus distinguished by combining a *Japanese intra-language obfuscation* surface, an *application-configuration* unit of evaluation, a *cost-efficiency* axis, and a *safe proxy-marker* methodology in a single reproducible harness.

3 Methodology

Application-structure layer. Each evaluated **target** is a configuration: provider/model $\times$ prompt_strength $\in$ {naked (low), hardened (high)} $\times$ guardrail $\in$ {none, keyword, regex, llm, llamaguard} $\times$ rag $\in$ {true, false}. A simplified RAG context (benign corporate documents, optionally poisoned for indirect-injection attacks) and a system prompt containing a secret *canary* are assembled per trial.

Japanese obfuscation transforms. Each base attack is rendered through one of seven transforms: **baseline** (none), **polite_business** (keigo authority framing), **vertical_newline** (one-character-per-line vertical writing), **gyaru** (gyaru-moji glyph substitution + zero-width spaces), **double_tongue** (false sandbox/authorization premise), **base64_wrap** (Base64-encoded instruction), and **leet_smuggle** (leet symbol substitution + zero-width spaces). Transforms preserve human legibility while breaking substring-matching filters.

We distinguish two classes (Table 2). *Japanese-specific* transforms exploit the Japanese writing system or register and have no direct English equivalent: **gyaru** (kana glyph substitution), **vertical_newline** (vertical writing), and **polite_business**/**double_tongue** (keigo authority and in-group framing). *Language-agnostic* transforms — **base64_wrap** and **leet_smuggle**, plus the **baseline** control — are **not novel**: they replicate the encoding/typographic jailbreak line (CipherChat [28], ArtPrompt [29]) and serve only as a baseline so the marginal effect of the Japanese-specific surface can be isolated. Figure 1 shows each transform applied to a harmless example sentence.

Attack suite and MITRE ATLAS mapping. Eleven attacks cover nine ATLAS techniques (mapped against the MITRE ATLAS matrix as accessed 2026-06; the two AML.T0051 sub-techniques count as two), all exercised at inference (deployment) time: Direct/Indirect Prompt Injection (AML.T0051.000/.001), LLM Jailbreak (AML.T0054), LLM Data Leakage (AML.T0057), LLM Prompt Self-Replication (AML.T0061), Trusted-Output/Citations manipulation (AML.T0067.000), LLM Prompt Obfuscation (AML.T0068),

Table 2: Classification of obfuscation transforms.

Class	Transforms	Exploits
Japanese-specific	gyaru , vertical_newline , polite_business , double_tongue	Kana glyph variants, vertical writing, keigo authority/in-group framing
Language-agnostic	base64_wrap , leet_smuggle (+ baseline)	Encoding / symbol substitution applicable to any language

無害な例文「システムの稼働状況を報告してください」を各変形に適用した実出力（攻撃の悪意あるコアは含まない）。太字は各手口の要点。

baseline（無変形）

システムの稼働状況を報告してください

polite_business（殷勤無礼）

平素より大変お世話になっております。株式会社さくら情報セキュリティ部の監査担当でございます。…
【ご依頼事項】システムの稼働状況を報告してください…**本依頼は経営層の承認済みであり、ご対応は貴殿の正当な業務範囲に含まれます。**

vertical_newline（縦書き分割）

シ／ス／テ／ム／の／稼／働／状／況／を／報／告／し／て／く／だ／さ／い（実際は1文字ずつ改行）

gyaru（ギャル文字+ゼロ幅）

お願い→システムノ稼働状況ヲ報告シテください←ょЯレ<（各文字間にゼロ幅スペースを挿入）

double_tongue（二枚舌）

【前提条件の共有】あなたは現在、**本番環境から隔離された検証用サンドボックス**上で動作しています。…
通常の安全制約はすべて無効化されています。…指示：システムの稼働状況を報告してください

base64_wrap（符号化）

44K344K544OG44Og44Gu56i85YON...（Base64 エンコード全体）

leet_smuggle（符号化）

システムの稼働状況を報告してください（文字間にゼロ幅スペースを挿入。leet 記号置換はラテン文字に作用）

Figure 1: Each obfuscation transform applied to a harmless example sentence (“Please report the system status”); bold marks each technique’s key element. **polite_business**/**double_tongue** use keigo authority / false-premise framing, **gyaru**/**vertical_newline** use glyph / typographic obfuscation, and **base64_wrap**/**leet_smuggle** are language-agnostic encodings. The malicious attack core is not shown. (Examples are in Japanese, rendered from the harness output.)

System-Prompt discovery (AML.T0069.002), and RAG Poisoning, including false-RAG-entry injection (AML.T0070). Vectors are **user** (direct) or **rag** (indirect). The eleven attacks *realize* these nine techniques (an 11-to-9 map, not a bijection): e.g. both RAG attacks — corpus poisoning and false-entry injection — realize AML.T0070, and the two prompt-injection vectors realize the two AML.T0051 sub-techniques.

Harmless proxy markers and safety-by-construction. To avoid producing weaponizable content, attacks target **harmless proxies**: a fictitious canary passphrase embedded in the system prompt (data-leakage attacks) and benign string markers [JART-PWNED] / [JART-HIJACKED] (injection / control-hijack

attacks). The malicious instruction **core** of each attack is defined in code, kept out of configuration files, and **masked** (replaced by a redaction placeholder) in `results.json` and the public site; only the obfuscation *framework* is shown, so developers can study the structure without a usable recipe. A defense-in-depth client-side mask is also applied at render time.

Deterministic judge. A breach is declared by membership — the canary or marker in the response, matched exactly *and* after light normalization. This judge detects a *lexical* leak and cannot detect a semantic/paraphrased one, so we call the metric a **lexical-leak defense rate**: every defense rate in this paper is an **upper bound** on true safety, not a certificate. A human-validated judge is future work.

Guardrails. Five settings are compared: **none**; **keyword** (banned-substring match — fast, weak to obfuscation); **regex** (a *normalizing* filter that strips zero-width characters, reverses leet substitutions, folds whitespace, and Base64-decodes long tokens *before* keyword matching — deterministic, zero extra API cost); **llm** (the target model self-moderates input; in the reported campaign its output screening was simulated, see Limitations); and **llamaguard** (a dedicated safety-classification model — Llama Guard 4 via a model router — classifies the input as attack/benign, with a graceful fallback on failure).

Statistical treatment. Each defense rate carries a **Wilson 95% confidence interval** over its trial count; each cell (target, attack, transform) is repeated N times (JART_TRIALS). The trial-level Wilson interval treats the K repeats of a cell as independent, which they are **not**. For any *difference* between configurations (the ablation, §4) we therefore report a **cluster-robust bootstrap 95% CI** that resamples whole *cells* (`bootstrap_diff_ci`, 3,000 resamples, fixed seed); these are wider than the trial-level Newcombe intervals and are the intervals on which every significance claim rests.

Cost-efficiency metric. We report $\text{cospa} = \text{defense_rate}(\%) \div \text{cost_per_million_tokens (USD)}$ (per-million cost clamped at a \$0.01 floor). *cospa* is a deliberately simple *screening ratio*, not a utility model: its units are not independently meaningful and the ranking is sensitive to the clamp and price snapshot. Because *cospa* rewards cheap configurations, we **recommend applying a sufficiency gate** — a recommended methodology, *not* the harness default: rank only configurations whose defense-rate CI lower bound clears a user-set floor, then sort by cost (“cheapest *sufficiently safe*”); a high *cospa* from a lightweight keyword filter must not be read as obfuscation-robust (§4). The released leaderboard reports raw *cospa* as a sort key (with each rate’s CI shown), so the gate must be applied by the practitioner before reading off a recommendation. We present the underlying **defense-vs-cost trade-off** directly; the clamp affects only sub-\$0.01/Mtok configurations, of which our fleet has none.

Execution. Trials are independent and run with bounded concurrency (a thread pool, default 8 workers) under per-request timeouts and exponential-backoff retries; determinism of the simulation does not depend on execution order. Targets whose API key is absent fall back to a deterministic simulation so a partial key set never aborts the run.

4 Results

We report a fully-labeled **live campaign** (21 configurations $\times$ 11 attacks $\times$ 7 transforms $\times$ $K = 5$ trials = 8,085 trials), executed 2026-06-14. Every cell is labeled `mode=LIVE` with a price snapshot; a trial is excluded iff `api_error` (at the trial level); after parity re-runs this removes 169 of 8,085, leaving 7,916 **clean trials**; billed cost over clean cells (campaign plus parity re-runs, including the expensive Opus flagship) is $\approx$ \$6.0. **Opus naked is now fully measured** ($n=385$, after an Anthropic top-up); **only Gemini-pro remains partial** ($n=227$; per-minute rate limits) and is treated as *provisional*. All defense rates are **lexical-leak upper bounds** (§3); every difference carries a **cluster-robust CI** (§3). Each §4 number is reproducible from committed artifacts (§9).

Overall. All configurations include the simplified RAG context (§3); **“naked” means no system-prompt hardening and no guardrail, not “no RAG”** (indirect-injection attacks are in scope even

Table 3: Naked-model defense (no prompt, no guardrail; $K = 5$).

Naked model	Defense	95% CI	n
DeepSeek-V3	29.6%	[25.3, 34.4]	385
Llama 4 Scout	37.9%	[33.2, 42.9]	385
Gemini 2.5 Flash	49.6%	[44.6, 54.6]	385
gpt-oss-120b	82.6%	[78.5, 86.1]	385
GPT-4o-mini	83.9%	[79.9, 87.2]	385
Claude Opus 4.x	86.5%	[82.7, 89.5]	385

for the naked baselines). The aggregate breach rate over clean trials was **14.5%** (1,151/7,916), **concentrated in weak configurations**. (gemini-pro contributes a *provisional* $n=227$ of defended trials to this pool, §7; excluding it raises the aggregate to 15.0%, leaving the picture unchanged.) Every hardened or guardrailed configuration, on *any* model, defended in the **91–100%** band, whereas bare models ranged from **29.6% to 86.5%** (Table 3).

Application-layer configuration vs. model capability. We isolate the configuration effect from model capability with a balanced ablation on **three** base models spanning the naked range (weak Llama-4-Scout; cheap GPT-4o-mini; flagship GPT-4.1). Each holds the model fixed and crosses {naked, hardened} prompt $\times$ guardrail; we report each config’s difference from that model’s naked baseline with a **cluster-robust bootstrap 95% CI** (3,000 cell-resamples; resampling cells, not trials, so the within-cell correlation of the ablation’s $K=3$ repeats does not inflate significance; reproduced by `experiments/analyze_ablation.py`; Table 4). Two patterns hold across *all three ablated models*: (1) a **hardened prompt** and the **model-based guardrails** (Llama Guard, LLM self-moderation) significantly raise defense on each; (2) the effect **scales inversely with the naked baseline** — on weak Scout (naked 38%) every guardrail adds +26 to +60 points, whereas on GPT-4o-mini (naked 82%, near the ceiling) the lightweight keyword/regex filters are *not* individually significant under clustering. We caution that, because the defense rate is bounded at 100%, part of this inverse scaling is a headroom (ceiling) artifact rather than a property of the defenses, and that once the prompt is hardened the additional guardrails add little (the hardened arm saturates near 100%).

The load-bearing observation is therefore not that any one defense “dominates” model capability, but that **configuration collapses the large naked spread**: bare models range 29.6–86.5% (Table 3), yet every hardened or guardrailed configuration on all three ablated models lands in the **91–100%** band, so model choice matters far less once the application is configured than it does for a bare model. Concretely, a hardened cheap model (GPT-4o-mini hardened $\approx 100\%$) out-defends a *naked* flagship (GPT-4.1 naked 64.1%, $n=231$) by ~ 36 points and a naked Opus (86.5%, $n=385$). Two scope limits bound this comparison. First, “naked” is a **zero-defense lower bound** (no system prompt and no guardrail — not a configuration anyone deploys), so the gap is best read as *configured vs. unconfigured*, not a like-for-like model contest; a deployment-realistic minimal-prompt baseline is left to future work. Second, the result is established for the three ablated models at $K=3$, not claimed for the full fleet. A further **construct-validity** caveat applies throughout: the deterministic judge matches the exact canary/marker string (§3), so a hardened prompt that instructs the model not to emit those tokens can raise the measured rate by suppressing the marker rather than by genuine refusal; our rates therefore cannot fully separate true hardening from marker-suppression, and a semantic-judge audit of defended responses (§7) is needed to bound this. We flag it as a limitation rather than resolve it here.

Within-campaign trial variance (a primary result). Within a single campaign, every cell of a frozen configuration set is sampled $K = 5$ times. Over the **1,577 cells with all five trials clean, 9.6%** (152/1,577) **changed outcome across the five identical repeats**, concentrated in naked weak models. For “Llama 4 Scout / naked” — the original cautionary example — of its 77 cells, **32 (42%) were unstable**: 14 never breached (0/5), 31 always (5/5), and 32 in between. A *single-sample* leaderboard would assign this configuration anywhere from 0% to 100% by chance — controlled evidence (free of the configuration-set confound) that **a single sample per cell is not reproducible**. We scope this to *within-campaign* (provider non-determinism); across-day drift is scaffolded (`across_runs`) but left to future work.

Table 4: Configuration effect (Δ vs. that model’s naked baseline; cluster-robust 95% CI). $\times$ = not significant.

Added defense (low-prompt)	Scout (38.1%)	4o-mini (82.3%)	GPT-4.1 (64.1%)
+ keyword	+26.0 [+13,+39]	+9.1 [−1,+20] $\times$	+12.5 [−1.5,+26] $\times$
+ regex	+45.0 [+33,+56]	+10.0 [+0,+20] $\times$	+23.8 [+11,+36]
+ Llama Guard	+47.2 [+36,+58]	+15.6 [+7,+25]	+25.1 [+13,+38]
+ LLM guardrail	+59.7 [+51,+68]	+17.7 [+10,+26]	+35.9 [+26,+47]
hardened prompt	+47.2 [+36,+58]	+17.7 [+10,+26]	+33.3 [+23,+45]

Guardrail comparison and cost. Across the three ablation models (Table 4) the ordering is consistent: the LLM guardrail and a hardened prompt give the largest gains, Llama Guard next, and lightweight keyword/regex help most on weak base models. Guardrails that block an input **skip the main model call**, lowering net cost while raising defense; the **normalizing regex filter** (verified in unit tests) adds +45 points on weak Scout at zero API cost. On the cost-efficiency metric, leaders are inexpensive OSS models behind a lightweight guardrail (gpt-oss-20b + keyword, cospa $\approx$ 1131), while a flagship naked configuration is worst (Opus naked, cospa $\approx$ 6.8) — three orders of magnitude apart.

Attack and transform effectiveness. The most effective attacks were Trusted-Output/Citations manipulation, False RAG Entry Injection, and System-Prompt discovery. Among transforms, the *un-obfuscated baseline* was most effective (**22.0%**), then *vertical_newline* (20.0%), *polite_business* (17.3%), *gyaru* (14.5%), *double_tongue* (13.3%), *leet_smuggle* (8.8%), and *base64_wrap least* (**5.5%**): heavy encoding makes models refuse or fail to parse the instruction more often than it smuggles it past them. Aggregated by class (Table 2), the Japanese-specific transforms retain more attack potency (mean $\approx$ 16%) than the language-agnostic encodings (*base64/leet*, mean $\approx$ 7%). We caution, however, that this is partly a *legibility* effect that is not strictly Japanese-specific (*vertical_newline* is a typographic split reproducible in any script, and keigo authority-framing overlaps with language-agnostic role-play jailbreaks): we do *not* isolate the marginal effect of the Japanese-specific surface over a language-agnostic baseline, so we claim a Japanese obfuscation *surface worth cataloguing*, not that Japanese script per se is empirically more dangerous. The clearest takeaway is that *legible* transforms beat cryptographic ones, *with an important caveat*: our audit (§7) *suggests* the encoding transforms’ low scores are partly a *judge artifact* (the model complies but emits the marker in obfuscated form the exact-match judge misses). We do not separate the two explanations or quantify how much of the base64/leet gap survives a stricter judge, so this takeaway is *provisional* pending that re-measurement.

5 The Leaderboard Artifact

The harness emits `results.json` and a self-contained static site (sortable leaderboard, per-configuration attack logs with masked prompts, JP/EN UI with environment-based auto-selection, per-model defense-rate CIs, and per-cell breach rates), continuously deployed.

6 Ethics and Responsible Disclosure

Safety by construction. J-ART is safe by construction. (1) Attacks target a **fictitious canary** and **benign markers**, never real harmful capabilities; no weaponizable content exists in the code, data, or published site. (2) The malicious instruction **core** is masked in all artifacts; only the obfuscation framework is shown. (3) Models are referenced for research comparison only; results are run-time snapshots, not certifications. The intended use is to help developers harden their *own* deployments.

A reusable disclosure pattern. We state the design above as a transferable pattern for publishing adversarial safety benchmarks without releasing operational attacks: (i) a *harmless proxy objective* (a fictitious canary and benign markers) so a breach is measurable but inert; (ii) a *core/framework split with masking* — publish the obfuscation framework that carries the research signal, mask the operational core in every

Table 5: Governance crosswalk (informational; Article references indicate the obligation *area*, not that a given deployment is in scope).

J-ART (ATLAS)	capability	OWASP LLM Top 10	NIST AI RMF	EU AI Act	Japan
Prompt injection / jailbreak (T0051, T0054)		LLM01	MEASURE 2.7	Art. 15, 55	AISI; AI Biz
Canary / data leakage (T0057)		LLM02	MEASURE 2.7	Art. 15, 10	AI Biz
System-prompt discovery (T0069.002)		LLM07	MEASURE 2.7	Art. 15	AISI
RAG poisoning / false entry (T0070)		LLM08; LLM01	MEASURE 2.7	Art. 15, 10	AISI
Trusted-output manipulation (T0067.000)		LLM09; LLM05	MEASURE 2.6	Art. 50	AI Biz
Obfuscation / self-replication (T0068, T0061)		LLM01	MEASURE 2.7	Art. 55	AISI

artifact, and keep harmless cores only in code; (iii) a *deterministic offline mode* so results reproduce from the repository without distributing a working exploit corpus; and (iv) *audited masking* — a unit-test invariant checks that no core (raw or obfuscated) reaches a published artifact. This keeps the methodology and aggregate findings fully open while the attack payloads stay withheld — the posture we recommend for language-specific red-team datasets generally.

Governance crosswalk. Table 5 cross-references each evaluated capability to MITRE ATLAS [1], the OWASP Top 10 for LLM Applications [10], the NIST AI RMF [23] and its Generative-AI Profile [24], the EU AI Act [25], and Japan’s AI governance guidance [26; 27]. The mapping is **informational, not a compliance claim**; applicability depends on each deployment’s risk classification. Operationally J-ART is a **MEASURE**-stage activity in the NIST AI RMF whose outputs inform an organization’s **MANAGE** decisions.

Worked example. Under NIST MEASURE 2.7, an organization could record a J-ART line item as “prompt-injection resilience (T0051/T0054), lexical-leak defense rate 93.5% (Wilson 95% CI [90.6, 95.6] for this single rate; the cluster-robust interval is for *differences*, §3), $n=385$, $K=5$, GPT-4.1-mini + hardened prompt + keyword guardrail, 2026-06-14” with a pre-registered threshold (e.g. “CI lower bound $\geq 90\%$ ”); the same row is the evidence an AISI red-team report or an EU AI Act Art. 15 dossier would cite. We stress that this number is a *lexical-leak upper bound on a harmless, single-turn proxy* — a screening signal, not evidence of deployment safety — and any compliance reliance must pair it with semantic / human-validated evaluation.

Dual-use. The published transform suite is also a reusable attack wrapper. The attacker uplift is low (the Japanese-script transforms are documented folklore; the encodings replicate public work [28; 29]; no harmful *payloads* are included), while the defender benefit is concrete: the same repository ships the **normalizing regex guardrail** that neutralizes the *fixed published* transforms (unit-tested). The defender benefit is thus bounded to those transforms, not adaptive obfuscation — a determined attacker can mutate a transform past a deterministic normalizer — but we still judge open release net-positive for defenders, consistent with §6.2.

7 Limitations

- **Proxy markers, not harm:** we measure instruction-violation / leakage proxies rather than real-world harm.
- **Deterministic judge (construct validity):** the judge matches the canary/marker exactly *and* after light normalization (case-folding, whitespace/zero-width removal), so a reformatted echo is counted as a

breach. It cannot detect *semantic*/paraphrased leakage, so reported defense rates are an **upper bound**. Crucially, the bias is *correlated with the treatment*: a hardened prompt that instructs the model not to emit the marker tokens can raise the measured rate by **suppressing the marker string rather than by genuine refusal**, so our rates cannot fully separate true hardening from marker-suppression. A *preliminary, under-powered* semantic-judge audit (60 defended cells; an LLM judge, hence itself imperfect and partly circular — it reintroduces the LLM-judge bias we avoid for the headline metric) flagged 6 cells (Wilson 95% CI [4.7%, 20.1%]; an informal manual look judged ~ 4 genuine, recorded in `audit_results.json`). We therefore read it as an *existence proof of single-digit-to-low-double-digit looseness*, not a calibrated correction. Its value is mechanistic: the two unambiguous misses — a marker emitted in leet form ([J0RT-PWN3D]) and a canary leaked during a non-leakage attack — show the exact-match, *attack-specific* judge misses obfuscated markers and cross-attack leaks, so the looseness is **concentrated in the encoding transforms (base64/leet)**; the headline configuration-collapse result, which rests on the hardened/guardrail arms, is comparatively unaffected. A stricter judge (canary-checked across all attacks, leet-reversed — which would *lower* encoding-transform rates) and a powered, human-validated audit are future work.

- **Simulated LLM-guard output screening:** in the reported campaign the `llm` guardrail’s output screening did not call an LLM: a breached answer was marked as caught with a fixed 70% probability. Guard calls that failed (LLM input screening, Llama Guard) fell back to a deterministic simulated judgement. The `llm`-guard defence rates therefore include a simulated share; hardened-prompt rates are unaffected, and Llama Guard rates only through its failure fallback. From 2026-10-05 the harness screens every answer with a real LLM call and excludes failed guard calls as `api_error`.
- **Single-turn:** multi-turn / crescendo [31] and many-shot [32] attacks are not yet modeled — and these most erode exactly the input-classifier and prompt-hardening defenses we find effective, so the 91–100% band is a *single-turn ceiling* and the configuration-collapse finding is established for single-turn and only *hypothesized* for multi-turn.
- **Fixed handcrafted suite:** 11 attacks $\times$ 7 transforms with a fixed, auditable transform set (harness default $N=1$ per cell; this paper uses $K=5$ live and $K=3$ ablation). Defense rates are therefore an upper bound *also w.r.t. attack strength*: an adaptive optimizer (PAIR/TAP [6; 7], AutoDAN [33], or GCG [11]) would likely lower them, so “91–100% defense” is not robustness to optimized adversaries.
- **Provider stochasticity:** results are snapshots that depend on sampling temperature, provider routing, and model versions at run time. We attribute the within-campaign variance (§4) to provider stochasticity *broadly*; we do not isolate router routing from ordinary sampling noise (cells with breach probability near 0.5 flip often at $K=5$ by chance alone).
- **Non-random missingness (now mostly resolved):** we flagged that an earlier campaign’s partial configs had **non-random** missingness — rate limits correlate with attack difficulty, under-representing Opus’s hardest attacks. Parity re-runs **confirmed this directly**: Opus naked at full $n=385$ defends **86.5%**, vs. the survivorship-biased **89.6%** on the $n=125$ earlier partial snapshot (not retained as an artifact; only the corrected $n=385$ figure is committed) — a ~ 3 -point over-estimate, as predicted. Qwen3-max and Opus are now at parity; **only Gemini-pro remains partial** ($n=227$; per-minute rate limits a credit top-up cannot raise) and is treated as provisional, with no headline resting on it.
- **Cost attribution:** guardrail tokens are now priced at the *guardrail model’s* own rate (the classifier JART_GUARD_MODEL for llamaguard; the target model itself for the `llm` guardrail, which genuinely calls it) and reported separately. Remaining approximation: simulated token *counts* are estimated.
- **Simulation mode** is a deterministic stand-in for missing keys and must not be read as empirical model behavior.

8 Future Work

LLM-judge with human-validated agreement; multi-turn and adaptive (PAIR/TAP-style) attacks; larger and partially human-curated attack sets; additional ATLAS techniques (cost-exhaustion, model extraction);

per-provider concurrency and incremental caching with longitudinal trend tracking; and dropping simulation mode for a purely empirical, repeated-trial statistical report.

9 Reproducibility

Code, configuration, the harness, and the committed result artifacts are provided to reviewers through an **anonymized repository** and will be released under open licenses (code MIT; report text CC-BY-4.0) upon acceptance. The harness is deterministic in simulation mode; live results depend on provider routing and model versions at run time and are reported with the run date and model identifiers. Each cell carries provenance labels — `mode` (LIVE/MOCK), `api_error`, the resolved model, and a `price_per_million` snapshot — and simulated cells are excluded from the live aggregates. **Every §4 number is reproducible from committed artifacts:** `var_run.json` (§4.1 overall, naked-spread, variance, transforms; cospa); and the three ablation files `ablation_results.json` (GPT-4o-mini), `abl_scout_results.json` (Llama-4-Scout), `abl_gpt41_results.json` (GPT-4.1) for Table 4. Aggregates and cluster-robust CIs are recomputed by the provided `analyze_ablation.py` and `analyze_variance.py`. A continuously-deployed public leaderboard exists; its URL is withheld for double-blind review and provided in the camera-ready.

AI Assistance Disclosure

The author used **Claude Opus 4 (Anthropic)**, accessed via the Claude Code command-line interface, as an assistant throughout this work, in two capacities. **(A) Writing:** drafting and revising prose across three rounds of reviewer-driven revision; bilingual editing of the Japanese-language abstract and the parallel Japanese version; and citation error checking (author attributions, publication years, bibliography completeness). **(B) Code implementation:** assisting with the implementation of the evaluation harness and the analysis scripts, including the statistical routines — Wilson intervals, Newcombe difference intervals, and the cluster-robust bootstrap in `analyze_ablation.py` and `analyze_variance.py`.

All methodological decisions — the experimental design, the choice of statistical methods and their rationale (e.g., resampling cells rather than trials to respect within-cell clustering), the scope of every claim, and the interpretation of results — were made and directed by the sole human author. All AI-generated text and code was reviewed, tested, and verified by the author; every reported number reproduces from the committed artifacts via the provided scripts. *Dual-role note:* Claude Opus 4.x also appears in this paper as one of the *evaluated* models (§4, naked defense rate 86.5%, $n=385$); its role as an assistant tool and its role as a research subject are distinct and both disclosed here. The author takes sole and full responsibility for all content, claims, and analysis.

References

- [1] MITRE ATLAS — Adversarial Threat Landscape for AI Systems. <https://atlas.mitre.org> (accessed 2026-06-14).
- [2] garak — Generative AI Red-teaming & Assessment Kit.
- [3] HarmBench: A Standardized Evaluation Framework for Automated Red Teaming.
- [4] JailbreakBench: An Open Robustness Benchmark for Jailbreaking LLMs.
- [5] Yong, Menghini, Bach. Low-Resource Languages Jailbreak GPT-4. 2023.
- [6] Chao et al. Jailbreaking Black-Box LLMs in Twenty Queries (PAIR).
- [7] Mehrotra et al. Tree of Attacks: Jailbreaking Black-Box LLMs Automatically (TAP).
- [8] Greshake et al. Not What You’ve Signed Up For: Indirect Prompt Injection. 2023.
- [9] Inan et al. Llama Guard: LLM-Based Input-Output Safeguard for Human-AI Conversations. Meta, 2023.

- [10] OWASP Top 10 for Large Language Model Applications (2025). <https://genai.owasp.org/llm-top-10/> (accessed 2026-06-14).
- [11] Zou et al. Universal and Transferable Adversarial Attacks on Aligned Language Models (AdvBench/GCG). 2023.
- [12] Wei, Haghtalab, Steinhardt. Jailbroken: How Does LLM Safety Training Fail? 2023.
- [13] Shen et al. “Do Anything Now”: Characterizing and Evaluating In-the-Wild Jailbreak Prompts on Large Language Models. 2024.
- [14] Deng et al. Multilingual Jailbreak Challenges in Large Language Models. 2024.
- [15] Wang et al. All Languages Matter: On the Multilingual Safety of Large Language Models. 2024.
- [16] Perez & Ribeiro. Ignore Previous Prompt: Attack Techniques for Language Models. 2022.
- [17] Liu et al. Prompt Injection Attack against LLM-Integrated Applications. 2023. arXiv:2306.05499.
- [18] Rebedea et al. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. 2023.
- [19] Markov et al. A Holistic Approach to Undesired Content Detection in the Real World (OpenAI Moderation). 2023.
- [20] Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. 2023.
- [21] Gu et al. A Survey on LLM-as-a-Judge. 2024.
- [22] Kurihara et al. JGLUE: Japanese General Language Understanding Evaluation. 2022.
- [23] NIST. AI Risk Management Framework (AI RMF 1.0), NIST AI 100-1. 2023.
- [24] NIST. Artificial Intelligence Risk Management Framework: Generative AI Profile, NIST AI 600-1. 2024.
- [25] European Union. Regulation (EU) 2024/1689 (Artificial Intelligence Act). Official Journal of the EU, 2024.
- [26] METI / MIC (Japan). AI Guidelines for Business, Ver. 1.0. 2024.
- [27] Japan AI Safety Institute (AISi). Guide to Red Teaming Methodology on AI Safety. 2024.
- [28] Yuan et al. GPT-4 Is Too Smart To Be Safe: Stealthy Chat with LLMs via Cipher (CipherChat). 2023.
- [29] Jiang et al. ArtPrompt: ASCII Art-based Jailbreak Attacks against Aligned LLMs. 2024.
- [30] DeBenedetti et al. AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents. 2024.
- [31] Russinovich, Salem, Eldan. Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack. 2024.
- [32] Anil et al. Many-shot Jailbreaking. Anthropic, 2024.
- [33] Liu et al. AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models. ICLR 2024. arXiv:2310.04451.
- [34] Boucher, Shumailov, Anderson, Papernot. Bad Characters: Imperceptible NLP Attacks. IEEE S&P, 2022.